

移动应用开发实验报告

封面信息

项目	内容
实验名称	跨平台应用开发
实验编号	d20301035103、d20301009203
学号	2023210594
姓名	赫宇豪
班级	软件 232
日期	2026.4.1
组别	组三 cifms
项目名称	云端智能畜牧养殖管理系统
技术栈	Uniapp

实验目的

- 1.掌握 4 种跨平台框架的列表页开发
- 2.学会 RESTful API 数据请求与 JSON 解析
- 3.了解移动设备传感器调用方法
- 4.完成框架适用性对比分析

实验环境

框架	开发工具	语言	依赖包
----	------	----	-----

框架	开发工具	语言	依赖包
Flutter	Android Studio/VS Code	Dart	http, provider
React Native	VS Code	JavaScript	axios, @react-native-community/refresh-control
Uniapp	HBuilderX	Vue	uni.request
MAUI	Visual Studio	C#	HttpClient, Newtonsoft.Json

实验步骤

1. 需求分析

1.1 数据模型定义

```
{
  "code": 200,
  "data": {
    "items": [
      {
        "id": 1,
        "title": "动物信息",
        "content": "牛 - 荷斯坦品种",
        "author": "管理员",
        "time": "2024-01-01 10:00:00"
      }
    ]
  }
}
```

1.2 API 接口规范

URL: http://10.89.4.150:3000

Method: GET

Response: JSON

2. Flutter 开发

2.1 项目创建

```
flutter create livestock_management
cd livestock_management
```

2.2 添加依赖

```
# pubspec.yaml
dependencies:
  flutter:
    sdk: flutter
  http: ^1.1.0
  provider: ^6.1.0
```

2.3 列表实现

```
// main.dart
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';
import 'package:provider/provider.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => LivestockProvider(),
      child: MyApp(),
    ),
  );
}

class Livestock {
  final int id;
  final String title;
  final String content;
  final String author;
  final String time;

  Livestock({required this.id, required this.title, required this.content,
    required this.author, required this.time});

  factory Livestock.fromJson(Map<String, dynamic> json) {
    return Livestock(
      id: json['id'],
      title: json['title'],
      content: json['content'],
      author: json['author'],
      time: json['time'],
    );
  }
}

class LivestockProvider extends ChangeNotifier {
```

```

List<Livestock> _livestockList = [];
List<Livestock> get livestockList => _livestockList;

Future<void> fetchLivestock() async {
  final response = await http.get(
    Uri.parse('http://10.89.4.150:3000')
  );

  if (response.statusCode == 200) {
    final data = json.decode(response.body);
    final items = data['data']['items'] as List;
    _livestockList = items.map((item) => Livestock.fromJson(item)).toList();
    notifyListeners();
  }
}

class LivestockListPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('动物管理')),
      body: Consumer<LivestockProvider>(
        builder: (context, provider, child) {
          return RefreshIndicator(
            onRefresh: () => provider.fetchLivestock(),
            child: ListView.builder(
              itemCount: provider.livestockList.length,
              itemBuilder: (context, index) {
                final livestock = provider.livestockList[index];
                return ListTile(
                  title: Text(livestock.title),
                  subtitle: Text(livestock.content),
                  trailing: Text(livestock.time),
                );
              },
            ),
          );
        },
      ),
    );
  }
}

```

3. React Native 开发

3.1 项目创建

```

npx react-native-init LivestockManagement
cd LivestockManagement

```

3.2 安装依赖

npm install axios @react-native-community/refresh-control

3.3 列表实现

```
// App.js
import React, { useState, useEffect } from 'react';
import { FlatList, RefreshControl, Text, View, StyleSheet } from 'react-native';
import axios from 'axios';

const App = () => {
  const [livestock, setLivestock] = useState([]);
  const [refreshing, setRefreshing] = useState(false);

  const fetchLivestock = async () => {
    try {
      const response = await axios.get('http://10.89.4.150:3000');
      setLivestock(response.data.data.items);
    } catch (error) {
      console.error(error);
    }
  };

  useEffect(() => {
    fetchLivestock();
  }, []);

  const onRefresh = async () => {
    setRefreshing(true);
    await fetchLivestock();
    setRefreshing(false);
  };

  const renderItem = ({ item }) => (
    <View style={styles.item}>
      <Text style={styles.title}>{item.title}</Text>
      <Text>{item.content} - {item.time}</Text>
    </View>
  );

  return (
    <FlatList
      data={livestock}
      keyExtractor={item => item.id.toString()}
      renderItem={renderItem}
      refreshControl={
        <RefreshControl refreshing={refreshing} onRefresh={onRefresh} />
      }
    />
  );
}
```

```

    );
  };

  const styles = StyleSheet.create({
    item: {
      padding: 16,
      borderBottomWidth: 1,
      borderBottomColor: '#eee',
    },
    title: {
      fontSize: 16,
      fontWeight: 'bold',
    },
  });
});

```

export default App;

4. Uniapp 开发

4.1 项目创建

打开 HBuilderX

新建 uni-app 项目，项目名称为“云端智能畜牧养殖管理系统”

4.2 列表实现

```

// pages/index/index.vue
<template>
  <view class="content">
    <view class="livestock-item" v-for="item in livestockList" :key="item.id">
      <text class="title">{{ item.title }}</text>
      <text class="info">{{ item.content }} - {{ item.time }}</text>
    </view>
  </view>
</template>

<script>
export default {
  data() {
    return {
      livestockList: []
    }
  },
  onLoad() {
    this.fetchLivestock();
  },
  onPullDownRefresh() {
    this.fetchLivestock().then(() => {
      uni.stopPullDownRefresh();
    });
  }
};

```

```

    },
    methods: {
        async fetchLivestock() {
            const res = await uni.request({
                url: 'http://10.89.4.150:3000'
            });
            if (res.data.code === 200) {
                this.livestockList = res.data.data.items;
            }
        }
    }
}
</script>

```

5. MAUI 开发

5.1 项目创建

打开 Visual Studio

新建.NET MAUI 项目

5.2 列表实现

```

// ViewModels/MainViewModel.cs
public partial class MainViewModel : ObservableObject
{
    [ObservableProperty]
    private ObservableCollection<LivestockItem> _livestockList = new();

    public async Task FetchLivestockAsync()
    {
        using var client = new HttpClient();
        var response = await client.GetAsync("http://10.89.4.150:3000");
        var json = await response.Content.ReadAsStringAsync();
        var data = JsonSerializer.Deserialize<LivestockResponse>(json);

        LivestockList = new ObservableCollection<LivestockItem>(data.Data.Items);
    }
}

```

6. 传感器集成扩展

6.1 GPS 定位实现（Flutter 示例）

```

import 'package:geolocator/geolocator.dart';

Future<void> getLocation() async {
    bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
    if (!serviceEnabled) return;
}

```

```
LocationPermission permission = await Geolocator.checkPermission();
if (permission == LocationPermission.denied) {
  permission = await Geolocator.requestPermission();
}

Position position = await Geolocator.getCurrentPosition();
print('位置: ${position.latitude}, ${position.longitude}');
}
```

实验结果

验证结果

框架	列表展示	网络请求	JSON 解析	下拉刷新	传感器集成
Uniapp	成功	成功	成功	成功	成功

截图记录

Uniapp 运行截图



框架对比分析

对比项	Flutter	React Native	Uniapp	MAUI
开发语言	Dart	JavaScript	Vue	C#
性能	★★★★★	★★★★☆	★★★★☆	★★★★☆
学习成本	中等	低	低	中等
生态成熟度	高	高	中	中

对比项	Flutter	React Native	Uniapp	MAUI
跨平台一致性	高	中	中	高
社区活跃度	高	高	中	中

问题与解决

问题描述	解决方案	解决结果
网络错误	修改 API 地址为局域网 IP	问题解决
图标不显示	移除图标配置，使用纯文字导航	问题解决
appid 不存在	在 HBuilderX 中重新获取 appid	问题解决
打包失败	安装项目依赖	问题解决
数据加载失败	启动本地 API 服务	问题解决

实验总结

实验收获

掌握了 Uniapp 框架的列表页开发流程

学习了 RESTful API 数据请求与 JSON 解析方法

实现了下拉刷新等用户交互功能

了解了跨平台应用的打包与部署流程

技术要点

Uniapp 开发经验：掌握了基于 Vue 的跨平台应用开发流程

网络编程能力：熟悉了 uni.request API 的使用方法

UI 设计能力：实现了响应式卡片布局 and 状态标识

打包部署经验：了解了 Uniapp 的云打包流程

考核指标

考核项	评分标准	得分
功能完整性	列表展示、网络请求、下拉刷新功能完整	25 分
代码质量	代码结构清晰，注释完整	20 分
多框架实现	至少完成 2 个框架的开发	25 分
传感器集成	成功集成传感器功能	15 分
框架对比	完成框架适用性分析	10 分
实验报告	报告结构完整，内容详实	5 分

教师评价

项目	评价
实验完成情况	
技术掌握程度	
创新点	
综合评价	
成绩	

教师签名：_____ 日期：_____